

Smallest enclosing balls by active-set methods, primal and dual

Thomas Schmelzer

September 4, 2026

Abstract

We record two active-set methods for the smallest enclosing ball problem, one on each side of it, and the invariance properties an implementation of either must preserve. The first ascends the dual — a quadratic program over the unit simplex — and holds no enclosing ball until it terminates; the second, of Fischer, Gärtner and Kutz, deflates an enclosing ball and is feasible throughout. Both terminate at an exact basis rather than at an interior-point tolerance, both cost one solve or projection of order at most d per iteration beside one sweep of the cloud, and both repair the same factorisation of the same support. What separates them is what they can hand back: dual weights that certify optimality in one pass, against a ball that is already valid at every step. On Gaussian clouds to $d = 1.6 \cdot 10^4$ they agree on the support set and differ by a small constant factor in time.

1 Introduction

Sylvester stated the problem in 1857 in a single sentence [10]: given a finite set of points in the plane, find the smallest circle containing all of them. It has been a standard example ever since — short enough to state in a line, geometric enough to draw, and just hard enough that the obvious algorithms are wrong. It is the 1-centre facility location problem, where one facility is placed so that the farthest customer travels as little as possible. It is the primitive under bounding-sphere hierarchies in graphics and collision detection. As the minimum enclosing ball of a set of feature vectors it is the core-set problem behind kernel machines. And it is a fixture of solver documentation: Hexaly ships it as a modelling template, in which the centre’s coordinates are the decision variables and the radius is an intermediate expression built from `sqrt`, `pow` and `max`, minimised directly [6]; CGAL ships it as `Min_sphere_d`, which is Welzl’s recursion with the move-to-front heuristic for small inputs and Gärtner’s method for large ones.

The algorithmic history is largely a computational-geometry one, and largely the work of two people. Welzl’s randomised incremental method [12] solves it in expected $O(n)$ time for fixed d , recursing over subsets with a few points forced onto the boundary; the constant grows superexponentially in d , and much of what followed is a response to that. Gärtner’s robust implementation [3] addressed the fragility of the recursion’s predicates in floating point, and is the other half of what CGAL runs. Gärtner and Schönherr [4] recast the problem as a quadratic program and solved it exactly, reserving judgment past a few hundred dimensions. Fischer, Gärtner and Kutz [2] then left the recursion behind altogether for a pivoting method that deflates an enclosing ball, carrying a dynamic QR of the support’s edge matrix and reporting d in the thousands. Elzinga and Hearn [1] had given a finite algorithm of that second kind two decades before Welzl.

What returned me to the problem was a pair of recent experiments with active-set methods elsewhere: the Goldfarb–Idnani dual method for strictly convex quadratic programs, whose invariants hold under bound constraints and quietly stop holding under general ones [8], and a conjugate-gradient solver wrapped in a primal–dual active-set loop for non-negativity constraints [9]. The smallest enclosing ball is that same machinery on the simplest feasible set there is. Its dual is a concave quadratic over the unit simplex; the active set is the set of points on the boundary sphere; and the certificate is geometric rather than algebraic — the points carrying weight are exactly the points at distance R , which a caller can check in one pass.

What follows records two methods on that structure, at equal length and with equal claim: the dual ascent of Section 3, and the primal deflation of Section 4, which is the pivoting method of Fischer, Gärtner and Kutz [2]. They keep the same support set, solve the same subproblem and repair the same factorisation (Section 5); they need the same care with tolerances (Section 6); and on every input measured in Section 8 they agree on the ball, on its support, and to within a small constant factor on time. Where they part is in what they hold while they run, and so in what they can promise if interrupted.

2 The problem and its two sides

For $S = \{p_1, \dots, p_n\} \subset \mathbb{R}^d$ the smallest enclosing ball is the solution of

$$\min_{x, r} r \quad \text{s.t.} \quad \|p_i - x\| \leq r, \quad i = 1, \dots, n, \quad (1)$$

a second-order cone program in $(r, x) \in \mathbb{R}^{1+d}$ with n blocks $(r, p_i - x) \in \mathcal{Q}^{d+1}$. Existence follows from coercivity of $f(x) = \max_i \|p_i - x\|$ and uniqueness from strict convexity of f^2 along any segment; we write (R, x^*) for the optimum.

Let $\Delta_n = \{u \in \mathbb{R}^n : u \geq 0, \mathbf{1}^\top u = 1\}$, and for $u \in \Delta_n$ put $x(u) = \sum_i u_i p_i$ and

$$\varphi(u) = \sum_i u_i \|p_i\|^2 - \|x(u)\|^2, \quad (2)$$

a concave quadratic. Everything below rests on one identity, valid for all $y \in \mathbb{R}^d$ and $u \in \Delta_n$:

$$\sum_i u_i \|p_i - y\|^2 = \sum_i u_i \|p_i\|^2 - 2x(u)^\top y + \|y\|^2 = \varphi(u) + \|x(u) - y\|^2. \quad (3)$$

Both steps are mechanical. The first expands the square and collects terms, using $\sum_i u_i p_i = x(u)$ on the cross term and $\mathbf{1}^\top u = 1$ on $\|y\|^2$; the second adds and subtracts $\|x(u)\|^2$, which by (2) leaves $\varphi(u)$ beside a completed square. What makes it an equality rather than a bound is that no term has been discarded: the weights enter only through $x(u)$ and through $\sum_i u_i \|p_i\|^2$, and φ is defined as exactly the difference of those two. So this is the parallel-axis identity of mechanics, or the bias–variance decomposition: measuring from anywhere other than the weighted mean costs precisely $\|x(u) - y\|^2$, whatever the weights, and $\varphi(u)$ is what is left when y is the mean.

Proposition 1. $R^2 = \max_{u \in \Delta_n} \varphi(u)$, and $x^* = x(w)$ for any maximiser w .

Proof. If $B(y, r) \supseteq S$ then the left side of (3) is at most r^2 , so $\varphi(u) \leq r^2$ for every u ; taking the optimum gives $\max \varphi \leq R^2$. Conversely let $T = \{p_i : \|p_i - x^*\| = R\}$. If $x^* \notin \text{conv } T$, separate: there is v with $v^\top(p - x^*) > 0$ on T , and $\|p - (x^* + tv)\|^2 = R^2 - 2tv^\top(p - x^*) + t^2\|v\|^2 < R^2$ for small $t > 0$, while the finitely many points off T stay strictly inside — so the radius decreases, a contradiction. Hence $x^* = \sum_i w_i p_i$ for some $w \in \Delta_n$ supported on T , and $y = x^*$ in (3) gives $\varphi(w) = \sum_i w_i \|p_i - x^*\|^2 = R^2$. The same substitution forces $\|x(w) - x^*\|^2 \leq 0$ for any maximiser. $\square$

Since $\partial\varphi/\partial u_i = \|p_i - x(u)\|^2 - \|x(u)\|^2$, the first-order conditions for $\max_{\Delta_n} \varphi$ read, after cancelling the common term,

$$\|p_i - x\| = R \quad (w_i > 0), \quad \|p_i - x\| \leq R \quad (w_i = 0), \quad (4)$$

the geometric certificate: the points carrying weight lie on the boundary sphere and the rest are enclosed. By Carathéodory the support may be taken of size at most $d + 1$, so (1) is determined by at most $d + 1$ of the n points and the problem is combinatorial in the choice of that subset.

Read forwards, (4) is a search for weights: keep a set of points on the sphere, adjust the weights on it, and stop when no point lies outside. Read backwards it is a search for balls: keep a ball that already contains S with a set T on its boundary, and shrink it until its centre falls inside $\text{conv } T$. The second reading rests on a fact due to Seidel, recorded in [4] and taken as Lemma 1 by Fischer, Gärtner and Kutz [2]: if T lies on the boundary of a ball B with centre c , then B is the smallest ball enclosing T if and only if $c \in \text{conv } T$. That is a stopping certificate for S as well. If in addition $S \subseteq B$ then $B \supseteq \text{seb}(S)$, while $T \subseteq S$ gives $\text{seb}(T) \subseteq \text{seb}(S)$; with $B = \text{seb}(T)$ the two inclusions close, and $B = \text{seb}(S)$ by uniqueness. So a ball containing the cloud, carrying T on its boundary, and centred inside $\text{conv } T$ is *the* answer — and $\text{conv } T$ is the only thing left to check. Sections 3 and 4 are those two readings made into methods — the second being theirs. They keep the same kind of state — a subset of S that will end up on the boundary, and a point in its affine hull — and differ in which feasibility they refuse to give up. The first stays dual feasible, so by Proposition 1 its radius rises to R from below and its ball encloses nothing until the last iteration; the second stays primal feasible, so its radius falls to R from above and every iterate is an answer, merely not yet the best one.

3 The dual active-set method

Maintain a working set F and weights $u > 0$ on it supported there and summing to one:

$$u \in \Delta_n, \quad \text{supp}(u) \subseteq F, \quad \varphi(u) \text{ non-decreasing}, \quad (5)$$

the dual counterpart of the invariant (9) carried by the method of Section 4. This is a primal active-set method run on the dual (2): the free variables are the weights of the support, every subproblem solves one face exactly, and feasibility for the original problem arrives only at the end — by Proposition 1, $\sqrt{\varphi(u)}$ is a lower bound on R at every iterate, so the ball on hand is too small until the last one.

Subproblem. Maximising φ over weights supported on F with no sign restriction is, by (4) without the inequalities, the requirement that x be equidistant from $\{q_j\}_{j \in F}$ and lie in their affine hull. With D the $(m-1) \times d$ matrix of edges $e_j = q_j - q_0$ and $x = q_0 + D^\top y$,

$$(DD^\top)y = \frac{1}{2}(\|e_1\|^2, \dots, \|e_{m-1}\|^2)^\top, \quad w = \left(1 - \sum_j y_j, y\right), \quad (6)$$

a dense system of order $m-1 \leq d$, non-singular exactly when F is affinely independent. This is the circumcentre of the face, expressed in barycentric coordinates.

Dropping. If $\min_j w_j < 0$ the circumcentre lies outside $\text{conv}\{q_j\}$ and the subproblem solution is dual-infeasible. Move along $s = w - u$ by the ratio test

$$\alpha = \min\{-u_i/s_i : s_i < 0\}, \quad (7)$$

which drives at least one weight to zero; remove it from F .

Degeneracy. If F is affinely dependent, $DD^\top$ is singular. The directions s with $\mathbf{1}^\top s = 0$ and $\sum_i s_i q_i = 0$ — equivalently $D^\top s_{1:} = 0$ after eliminating s_0 — leave $x(u)$ fixed, so $\|x(u)\|^2$ is frozen and $\varphi(u + ts) = \varphi(u) + t \sum_i s_i \|p_i\|^2$ is linear: the subproblem is unbounded. Project the gradient onto that null space and apply (7), shrinking F by one.

Adding. Otherwise accept $u \leftarrow w$, set $x = x(u)$ and $R = \max_{i \in F} \|p_i - x\|$, and let $k = \arg \max_i \|p_i - x\|$. If $\|p_k - x\| \leq R$ then (4) holds and (R, x, u) is optimal; otherwise append k to F with weight zero. By (8) below this is also the steepest-ascent choice among the fixed variables.

Input $p_1, \dots, p_n$. Recentre on $\bar{p}$. Set $F = \{k\}$, k farthest from p_1 ; $u_k = 1$.
Repeat: if F affinely dependent, take the null-space descent step and drop; else solve (6). If $\min w < 0$, take the ratio step (7) and drop. Else set $u = w$, $x = x(u)$, $R = \max_{i \in F} \|p_i - x\|$; if no point exceeds R , return $(R, x + \bar{p}, u)$; else drop zero weights and append $\arg \max_i \|p_i - x\|$ with weight 0.

Each iteration either strictly increases φ or strictly shrinks F , and there are finitely many working sets, so the method is finite; as with any active-set method a degenerate zero-length step requires an anti-cycling rule in exact arithmetic — for this problem, a Bland-type order on the points, which Fischer, Gärtner and Kutz [2] prove terminating and then reject as too slow. In practice an iteration cap serves as a safety net. Note that

$$\partial\varphi/\partial u_k - \partial\varphi/\partial u_i = \|p_k - x\|^2 - \|p_i - x\|^2, \quad (8)$$

so the farthest violator is the fixed variable with the largest reduced gradient.

4 The primal pivoting method of Fischer, Gärtner and Kutz

The method of Fischer, Gärtner and Kutz [2] carries a ball rather than a set of weights, and never surrenders it. Its state is a pair (c, T) , a centre and a support set, subject to the invariant of their Fig. 2:

$$S \subseteq B(c, T), \quad T \subseteq \partial B(c, T), \quad T \text{ affinely independent}, \quad (9)$$

where $B(c, T)$ is the ball about c through the farthest point of T . By Seidel's lemma the pair is optimal as soon as $c \in \text{conv } T$, and c is then the circumcentre $\text{cc}(T)$ without further argument — it lies in $\text{aff}(T)$ and (9) has it equidistant from T . So the loop runs until c reaches $\text{conv } T$. Each iteration is a dropping phase, entered only once c has landed in $\text{aff}(T)$, followed by a walking phase.

Dropping. $c \in \text{aff}(T) \setminus \text{conv } T$ means some affine coefficient of c with respect to T is negative; drop such a point. The coefficients are the same object as the barycentric weights of (6) and come from the same factorisation (Section 5), but here they are read rather than moved: there is no ratio test, because c does not travel during a drop. Their Lemmata 4 and 5 together say the dropped point cannot stop the walk that immediately follows, which is what keeps the two phases from undoing each other.

Walking. Move c along $e = cc(T) - c$. Their Lemma 3 says that segment is orthogonal to $\text{aff}(T)$, that T stays on the boundary the whole way, and that the radius strictly decreases — the deflation. Two consequences make the step cheap. Orthogonality makes $cc(T)$ the projection of c onto $\text{aff}(T)$, so no circumsphere is ever solved for and one projection does the work of (6). And the stopping fraction has a closed form: with $c(a) = c + ae$, a point p joins the boundary when $\|p - c(a)\|$ equals the common support distance; the a^2 terms cancel, and $\langle q - c, e \rangle = \|e\|^2$ for every $q \in T$ leaves

$$a_p = \frac{R_c^2 - \|p - c\|^2}{2(\|e\|^2 - \langle p - c, e \rangle)}, \quad R_c = \max_{q \in T} \|q - c\|, \quad a = \min\{1, \min_p a_p\}. \quad (10)$$

Criterion (1) of Lemma 4 — that p lies behind $\text{aff}(T)$ and so cannot stop the walk — is exactly the statement that this denominator is non-positive, so one sign test separates the candidates and the numerator is non-negative on the rest by (9). If $a = 1$ the centre arrives at $cc(T)$, lands in $\text{aff}(T)$, and the next iteration drops; otherwise the minimising point enters T and the ball has shrunk.

Pivot rules. Ties are resolved two ways. *Bland’s rule*, adapted: fix an order on S , drop the negative-coefficient point of smallest rank, and admit the smallest-rank point among those that stop the walk together. Their Theorem 1 proves termination with it, degeneracies included. The *heuristic*, which is what their own code runs and what the tables below report, drops the *most* negative coefficient and admits the tied point farthest from $\text{aff}(T)$ — the enlargement that leaves the support best conditioned. The paper records Bland’s rule as correct but slow.

Input $p_1, \dots, p_n$. Recentre on $\bar{p}$. Set $c = p_1$ and $T = \{k\}$, k farthest from c .
Repeat: if $c \in \text{aff}(T)$, take the affine coefficients of c ; if none is negative, return $(\max_{q \in T} \|q - c\|, c + \bar{p}, T)$; else drop one negative-coefficient point by the pivot rule. Otherwise walk: form $e = cc(T) - c$ as the projection of c onto $\text{aff}(T)$, take a from (10) over the points with positive denominator, set $c \leftarrow c + ae$, and if $a < 1$ admit the minimising point into T .

The radius falls strictly on every walk and no configuration recurs, so the method is finite; under Bland’s rule that is Theorem 1, and under the heuristic an iteration cap serves as the safety net, exactly as in Section 3.

Degeneracy, and the price of the invariant. The invariant (9) requires T affinely independent, and there is no fallback if it is not: with $\text{aff}(T)$ of the wrong dimension the projection that defines the walk is meaningless. So the entering point must be certified off $\text{aff}(T)$ before it is admitted, which is the paper’s stability threshold and the one tolerance in the method carrying units of length (Section 6). Section 3 makes the opposite choice: it *permits* a dependent support and answers it with a descent direction in the null space, which costs it a rank test on every iteration and a factorisation that must survive a state an economic QR cannot represent. Neither choice dominates — one avoids the case, the other resolves it.

How the two differ. The support sets agree, the drop criteria agree — a negative barycentric coordinate, in both — and the mechanics agree nowhere else. Section 3 takes the whole step to $cc(F)$ and only then consults the data, admitting the farthest violator; this method never takes a full step while any point is still outside, truncating the walk at whichever point first touches the shrinking sphere, which is also the point that enters. A drop here is immediate and followed by a resumed walk; there it is a ratio test in weight space, moving every weight at once. And the certificate differs in kind: Section 3 finishes with weights that verify (4) in one pass, while this method finishes with what it has held all along, a ball that contains S . One of the paper’s observations deserves flagging in that light: almost-cospherical clouds are their *hardest* inputs, points entering and leaving T repeatedly, whereas cospherical inputs are where the dual method is at its most accurate. Both can hold at once — theirs is a claim about iteration counts, the other about the final answer.

5 The support’s factorisation

Per iteration, both methods want the same two things from their support: whether it is affinely independent, and where the circumcentre of its face lies. Both read those off the same object. Let D be the $r \times d$ matrix of edges $q_j - q_0$ of the current support and let $D^\top = QR$ be its economic factorisation. Three identities then take every per-iteration quantity off d and onto the $r \times r$ block:

$$DD^\top = R^\top R, \quad \ker D^\top = \ker R, \quad \|q_j - q_0\|^2 = \|Re_j\|^2. \quad (11)$$

The first makes (6) two triangular solves against a Cholesky factor that is already to hand, with no $O(r^2d)$ product; the second makes the affine-dependence test of Section 3 a rank test on R — note it is the *right* null

space, not the one the edge matrix itself would hand you; the third supplies the right-hand side. The pivoting method reads the same R for its projection and its coefficients.

Rebuilding that factorisation costs $O(dr^2)$ and repairing it costs $O(dr)$, and the repair is section 4 of [2]. Three updates cover every move either method makes: a point joining the support appends a column, a point other than q_0 leaving deletes one, and q_0 leaving has no column of its own — every column being measured *from* it — so q_1 is promoted by a deletion followed by a rank-one update. In a compiled language these are Givens sweeps; in an interpreted one they have to be the library's, or a single vectorised refactorisation will beat them and the measurement will be of the interpreter rather than of the data structure.

Two things keep this from being a pure win. The dependent supports Section 3 permits are exactly what an economic QR cannot hold — an update refuses a column already in the span — so those faces are met by refactorising, which is affordable only because the stability of the surrounding tolerances keeps them rare. And the repair is not cheaper at every size: measured on the clouds of Section 8 it is worth a factor of 3.5 at $d = 8000$, is at parity near $d = 100$, and loses below that, where the per-update overhead exceeds the small dense solve it replaces. An implementation therefore dispatches on d rather than committing to one route. That threshold is why Section 8 can report $d = 1.6 \cdot 10^4$ at all, and why the Gaussian clouds of Table 2 are indifferent to it.

6 Invariance and floating point

Two properties are load-bearing for either method and easily lost by both. Both are consequences of refusing any tolerance tied to coordinates of magnitude one.

Origin invariance. Recentre on $\bar{p}$ before iterating, and form $\|p - x\|^2$ as $\sum_k (p_k - x_k)^2$ rather than $\|p\|^2 - 2p^\top x + \|x\|^2$. The expanded form cancels quantities of size $\|x\|^2$ to produce an answer of size R^2 , losing every digit by which $\|x\|$ exceeds R ; for a cloud of unit extent at distance 10^6 the rounding floor of a squared distance is then of the order of the radius itself.

Scale invariance. Test affine dependence on the rank of D , not on $[1; Q^\top]$: the row of ones has unit entries, dominates the singular values, and would declare any cloud of extent below ε degenerate. Size the feasibility slack of the stopping test as a relative tolerance plus an absolute floor proportional to $\varepsilon \max_i \|p_i\|^2$, and normalise the null-space direction before comparing anything derived from it — the gradient carries units of length², the weight tolerance does not.

Which tolerance carries units. The two methods have one apiece, and they are not the same one. Section 3 must decide when a weight has reached zero, which is dimensionless, and when a null-space direction is binding, which is not — hence the normalisation above. Section 4 must decide when a candidate is far enough off $\text{aff}(T)$ to be admitted, which carries units of length and is sized off the cloud's extent; its other tolerance, on the affine coefficients, needs no such care, being dimensionless and bounded by $1/2$ in the optimal configuration (Lemma 2 of [2]). The pattern is the same in both: the dimensionless test gets a constant, the dimensioned one gets the extent.

Exponent range. Tolerances are not the whole of it. A double reaches to 10^{308} , so a *squared* double reaches only to 10^{154} , and both methods square everything: the Gram matrix, the distances, the noise floor. A cloud of extent 10^{-160} — ordinary coordinates — therefore has a Gram matrix in the subnormals, and the solve (6) overflows; at 10^{+160} the squares saturate and a method reports $R = 0$ without raising. Rescaling the centred cloud to unit extent removes both, and the factor must be a power of two: any other rounds, and the exactness Table 1 reports is lost with it. The measurements below rescale in the dual method and not in the pivoting one, which is a property of the two implementations rather than of the two methods — the walk of (10) would survive the same treatment unchanged.

7 Cost, and what each method certifies

Per iteration each method performs one $O(nd)$ sweep of the cloud — squared distances to the current centre in Section 3, the stopping fractions (10) in Section 4 — and one solve or projection of order at most d , repaired rather than rebuilt above the dimension Section 5 names. Nothing of size n is ever factorised by either. This is the structural difference from an interior-point method applied to (1), which factors an $n(d+1)$ -row system at every step. Both also identify the support combinatorially and solve that face exactly, rather than approaching it from the interior and leaving the caller to decide which slacks of size 10^{-9} were meant to be zero.

What they can hand back differs, and it is the practical reason to keep both. Section 4 is primal feasible at every step: interrupt it at any iteration and what is on hand is an enclosing ball, correct if not yet smallest, with a radius that only falls from here. That is the guarantee to want under a time budget, and Section 3 cannot offer it — by Proposition 1 its radius is a lower bound throughout, so its ball fails to contain the cloud until the final iteration. In exchange it finishes with the dual weights.

The weights are the conic dual of (1) after one rescaling: the block for point i is

$$z_i = u_i \left(1, -\frac{p_i - x}{R} \right) \in \mathcal{Q}^{d+1}, \quad (12)$$

whose tail has norm $u_i \|p_i - x\|/R$, equal to its head exactly on the support, and which pairs with the slack $s_i = (R, p_i - x)$ as $s_i^\top z_i = (u_i/R)(R^2 - \|p_i - x\|^2) = 0$ — complementarity, each factor vanishing where the other does not. So a solver built on this method returns a standard-form primal–dual pair, and a caller can verify optimality from (4) at the cost of one pass over the data, without re-solving.

Remark (Substitutability). This is what makes the method usable behind a conic solver’s interface rather than only as a special-purpose routine. The accompanying implementation dispatches on the cone list: zero and non-negative blocks go to a dense dual active-set QP method [5], and equally sized second-order blocks matching the shape of (1) go to Section 3, whose weights are what the interface asks for; Section 4 answers the same program but would have to solve (6) once at the end to produce them. A program carrying the right cones but a different matrix is refused rather than answered, which requires reconstructing the point cloud from (A, b) and rebuilding the program for comparison. An indefinite objective is likewise refused rather than regularised into definiteness, since $P + \epsilon I$ answers a different question.

8 A numerical comparison

Table 1 runs the two methods of Sections 3 and 4 beside two references: Clarabel 0.11.1 applied to (1), and Welzl’s algorithm [12]. All four compute the same object on the same Gaussian clouds. Two quantities are reported, because either alone would mislead.

The first is the relative enclosure error $\max_i \|p_i - x\|/R - 1$, where a positive value means the returned ball does not in fact contain the cloud. It is a deterministic property of the returned pair, free of timing noise. Here the three combinatorial methods are indistinguishable — all come in at zero or a single ulp, against Clarabel’s interior-point residuals three to five orders larger — because each returns a radius that is by construction the maximum distance over its own support. The exactness is thus a property of terminating at a basis, and neither of the two methods here has any of it to itself.

One ulp of that is worth naming, since it is the only place the three separate, and it is a difference between the two methods of this note. Section 3 solves afresh for the circumcentre of its final support and lands on zero; Section 4 reports the running sum of the walks that carried its centre there, and so shows a single positive ulp on four of the seven rows. Both identify the same support set. The difference is in the arithmetic that produces the centre, not in the combinatorics that select it, and at $2.2 \cdot 10^{-16}$ it is of no practical consequence — it is recorded only because a column of exact zeros beside a column of ulps invites the wrong inference.

The second is wall-clock time, and it must be read as a comparison of *implementations*. Both methods of this note spend their time in one vectorised sweep per iteration, whereas Welzl’s recursion evaluates a containment predicate per step in scalar Python; a good deal of the gap in the t columns is CPython against NumPy rather than one algorithm against another. A step count is therefore given alongside each combinatorial method that has an informative one: the circumspheres Welzl computes, and the pivot steps the walking method takes. Both are properties of the algorithm and the input in any language — but they count different things, a solve over at most $d + 1$ points against a projection plus a sweep over all n , and so track each method against itself rather than against the other.

n	d	enclosure error				time (s)				steps	
		act. set	pivoting	Clarabel	Welzl	act. set	pivoting	Clarabel	Welzl	pivots	bases
10^3	2	0	$-1.1 \cdot 10^{-16}$	$5.3 \cdot 10^{-12}$	$2.2 \cdot 10^{-16}$	0.000	0.000	0.008	0.010	3	265
10^3	5	$-2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$	$-7.2 \cdot 10^{-12}$	$2.2 \cdot 10^{-16}$	0.000	0.001	0.032	0.018	21	592
10^4	3	0	0	$3.8 \cdot 10^{-12}$	0	0.001	0.001	0.175	0.452	10	2406
10^4	20	0	$2.2 \cdot 10^{-16}$	$1.0 \cdot 10^{-12}$	—	0.006	0.015	0.983	—	90	—
10^5	2	0	$2.2 \cdot 10^{-16}$	$-8.0 \cdot 10^{-13}$	0	0.003	0.007	1.524	0.953	5	813
10^5	3	0	0	$1.3 \cdot 10^{-10}$	0	0.007	0.010	2.016	4.031	10	5998
10^5	10	0	$2.2 \cdot 10^{-16}$	$-1.3 \cdot 10^{-12}$	—	0.020	0.051	7.168	—	43	—

Table 1: Standard normal clouds, one per row; median of three runs. Apple M4 Pro, CPython 3.12.13, NumPy 2.5.1, Clarabel 0.11.1 at default tolerances. Welzl was not attempted on the rows marked —; Table 2 measures that limit. The pivoting method runs the heuristic pivot rule of [2], with the QR of the edge matrix rebuilt each pivot rather than updated.

Table 2 isolates the dependence on d at fixed n . Welzl’s median basis count rises from 191 to 275 574 between $d = 2$ and $d = 11$, a factor between 1.5 and 2.7 per dimension across the upper half of that range,

d	2	3	4	5	6	7	8	9	10	11
Welzl bases, median $/10^3$	0.19	1.05	2.50	6.4	12.0	21.4	58.6	115	189	276
Welzl bases, worst $/10^3$	0.55	1.70	4.92	12.9	35.9	49.3	194	577	864	1602
pivots, median	5	6	8	10	15	11	18	18	18	21
pivots, worst	6	15	11	18	25	23	33	34	36	52
Welzl time (s)	0.007	0.026	0.053	0.143	0.236	0.491	1.181	2.675	3.796	5.692
pivoting time (ms)	0.2	0.3	0.4	0.5	1.0	0.8	1.3	0.9	0.9	1.1
active-set time (ms)	0.2	0.3	0.3	0.4	0.5	0.5	1.0	0.6	0.6	0.6

Table 2: Growth in d at $n = 10^3$: medians over 15 seeds, worst seed on the second line of each pair. Medians are necessary — the basis count is high-variance, and a mean over five seeds still puts $d = 11$ below $d = 10$.

d	steps			enclosure error		time (s)			
	support	act. set	pivoting	act. set	pivoting	act. set	pivoting	ratio	
1 000	92	92	197	$-4.4 \cdot 10^{-16}$	0	0.148	0.232	1.56	
2 000	124	128	208	$-1.1 \cdot 10^{-16}$	$-1.1 \cdot 10^{-16}$	0.302	0.355	1.18	
4 000	177	177	249	$-6.7 \cdot 10^{-16}$	0	1.296	1.450	1.12	
8 000	217	217	328	$-1.0 \cdot 10^{-15}$	0	2.760	3.272	1.19	
16 000	283	283	352	$-1.4 \cdot 10^{-15}$	0	7.869	8.457	1.07	

Table 3: Growth in d at $n = 10^3$: standard normal clouds, one per row, median of three runs. Same machine as Table 1, with SciPy 1.18.0. Both methods repair the QR of their support’s edge matrix rather than rebuilding it, and both identify the same support set, whose size is the first column. *Steps* are active-set iterations against pivot steps; unlike the pair in Table 2 these two count comparable work, one solve or projection of order r plus one $O(nd)$ sweep each. Welzl’s method and Clarabel are not run here: neither finishes in this regime.

while both simplex-like methods stay under a millisecond throughout: their work per iteration is a solve or a projection of order at most d , and d is small.

The two step counts are what the sweep is really for, and they part company completely. Welzl’s grows by three orders of magnitude over these ten dimensions; the pivot count grows from 5 to 21, by a factor of four. That is the separation the pivoting method was built for, and it is visible here at $d \leq 11$ even though the regime it was built for begins far higher. The worst-seed lines matter as much as the medians. At $d = 11$ one of fifteen clouds cost 1.6 million basis computations, six times the median, where the worst pivot count is 52 against a median of 21 — a factor of 2.5. Both distributions have a tail that a single measurement will not find, which is the practical form the expected-time analysis takes; the tails are simply not the same size.

Table 3 turns both of those tables around: $n = 10^3$ fixed and d from 10^3 to $1.6 \cdot 10^4$, which is the regime [2] was written for and the first one in which the linear algebra rather than the combinatorics decides the cost. Neither of the other two references appears in it. Welzl’s median basis count is already $2.8 \cdot 10^5$ at $d = 11$, and Clarabel would be handed a program of $n(d+1) \approx 1.6 \cdot 10^7$ rows to factor. Both methods that remain repair the factorisation of their support rather than rebuilding it, as Section 5 describes, which is what lets either of them appear in this table at all.

The step counts are comparable in this table in a way the two of Table 2 are not: an active-set iteration and a pivot step each cost one solve or projection of order r plus one $O(nd)$ sweep. They separate by about a factor of two, in the direction the mechanics predict. The active-set count equals the final support size on four of the five rows — on Gaussian data the method adds points and essentially never drops one, so it spends one iteration per support point, the $d = 2000$ row costing four extra iterations for four drops — while the pivoting method takes 1.2 to 2.1 times as many steps. That is the price of never taking a full step: each walk is truncated by the first point of S to touch the shrinking sphere, so a point enters after a partial move rather than after a jump to $cc(F)$. The wall clocks do not separate by anything like that factor. Their ratio sits between 1.1 and 1.6 with no trend in d , and repeat runs of one row move it by a few percent, so it should be read as a small constant factor rather than as a curve: with r in the hundreds both methods spend most of each iteration in the same two places, the sweep over the cloud and the repair of the factorisation, and the pivoting method’s extra steps are its cheapest ones.

Two further things in the table are worth naming. The support size grows like $\sqrt{d}$ on these clouds — 92 points at $d = 10^3$ and 283 at $d = 1.6 \cdot 10^4$, a factor near $\sqrt{2}$ per doubling — so it stays an order of magnitude or more below the Carathéodory bound $d + 1$, and the per-iteration solve is of that order rather than of d . With $O(\sqrt{d})$ iterations of $O(nd)$ each the total should scale as roughly $d^{3/2}$, and it does: a 53-fold increase in time over a 16-fold increase in d . And the enclosure errors stay at a few ulp, for both methods, across four doublings of d — so the exactness of Table 1 is a property of terminating at a basis, not of small d .

Read against the paper being compared to, Table 3 runs to $d = 1.6 \cdot 10^4$, past the 2000 [2] reports, with the same support on every row. Table 1 is the opposite regime and the one their analysis says least about: their per-pivot cost does not fall with n while the $O(nd)$ sweep vectorises, and there too the two are separated by a small constant factor rather than by anything structural, their pivot counts running from 3 to 90.

None of the three comparisons is fair to its subject, and the third is the least fair. Clarabel is built for large sparse programs of arbitrary shape, and Welzl’s recursion is being run in the least favourable language for one. The pivoting method carries a narrower handicap, and only in Tables 1 and 2: its columns there were measured with the QR of the edge matrix rebuilt at every pivot, where [2] updates it by Givens rotations in $O(d^2)$. At $d \leq 20$ that costs it little, the factorisation being of order at most 21, which is why those two tables were left as measured and Table 3 — where the data structure is the whole point — runs both methods with it in place. At $d = 20$ the pivoting method takes 2.5 times the active-set time; nothing in that ratio should be carried to the $d \sim 10^3$ regime their paper reports, and Table 3 is what happens there instead. That the specialised method wins on its own problem, in its own regime, against implementations chosen for clarity is the weakest possible claim, and the only one made here.

9 Relations to neighbouring methods

Welzl’s algorithm

Welzl’s randomised incremental method [12] computes the same object and, as Table 1 confirms, terminates at an exact basis just as the two methods above do. It differs from both in how that basis is searched.

Welzl’s search is combinatorial: it recurses over subsets, drawing a point p at random, solving without it, and re-solving with p forced onto the boundary when p falls outside the ball just computed. The permutation drives the recursion, the only arithmetic is the primitive that circumscribes at most $d + 1$ points, and the analysis is that of LP-type problems — expected $O(n)$ time for fixed d , with a constant known to grow superexponentially in d . The method of Section 3 searches instead by ascent on φ : the weights choose the pivot, a negative barycentric coordinate calling for a drop and the farthest violator for an addition. The search is monotone, local, deterministic, and admits add *and* drop steps; what it does not admit is an expected-time bound, and Section 3 claims none.

Table 2 is the practical shape of that difference: at fixed n the recursion’s cost climbs by a factor of two or so per dimension and carries a long upper tail, while a solve of order d does not notice d in this range. The other difference is the certificate. Welzl’s recursion yields the ball and its basis; the method above yields the barycentric weights as well, which are what make (4) checkable in one pass and what become (12) when a conic interface asks for a dual. The weights can be recovered from a basis by solving (6), but they are not a by-product of the search there, whereas here they *are* the search — which is also why a support set warm starts a perturbed cloud and a fresh random permutation cannot.

Gärtner’s robust implementation [3] addresses the fragility of the recursion’s predicates; Section 6 answers the same question here.

The minimum-norm point problem

If all points have equal norm ρ — after the standard lift of a kernel problem onto a sphere, say — then (2) reduces to $\varphi(u) = \rho^2 - \|x(u)\|^2$, and maximising it over Δ_n is minimising $\|x\|$ over $\text{conv } S$: the minimum-norm point problem of Wolfe [13] and of the Frank–Wolfe family [11]. The working set is then Wolfe’s corral, the subproblem (6) his affine minimiser, the drop step his major cycle. Restoring the linear term $\sum_i u_i \|p_i\|^2$ gives the general case, and is what makes the subproblem a circumcentre rather than a projection.

10 Notes

Of the two exact lines of work Section 1 sketches, the method of Section 4 is the geometric one and that of Section 3 the quadratic-programming one, after Gärtner and Schönherr [4], whose reservation that a QP formulation needed exact arithmetic past a few hundred dimensions Table 3 bears on without settling: in double precision the method returns a ball out to $d = 1.6 \cdot 10^4$ whose enclosure error is a few ulp, on Gaussian clouds, which are not the inputs that reservation was about. The QP machinery is standard [7]. An implementation of the method, of the pivoting method of Section 4, of the Goldfarb–Idnani core, and of the conic front end is available at <https://github.com/tschm/cvxball>, where the two methods this note compares ship side by side and the Clarabel and Welzl routes of Table 1 sit beside them as references.

References

- [1] J. Elzinga and D. W. Hearn. Geometrical solutions for some minimax location problems. *Transportation Science* 6(4):379–394, 1972.
- [2] K. Fischer, B. Gärtner and M. Kutz. Fast smallest-enclosing-ball computation in high dimensions. *Proc. ESA*, 630–641, 2003.
- [3] B. Gärtner. Fast and robust smallest enclosing balls. *Proc. ESA*, 325–338, 1999.
- [4] B. Gärtner and S. Schönherr. An efficient, exact, and generic quadratic programming solver for geometric optimization. *Proc. 16th Annu. ACM Sympos. Comput. Geom.*, 110–118, 2000.
- [5] D. Goldfarb and A. Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming* 27:1–33, 1983.
- [6] Hexaly. Smallest circle problem. Modelling template, <https://www.hexaly.com/templates/smallest-circle-problem>, accessed September 2026.
- [7] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006.
- [8] T. Schmelzer and M. Stoll. Goldfarb–Idnani revisited: invariants, certificates, and the limits of guessing. arXiv:2608.30933, 2026.
- [9] T. Schmelzer and M. Stoll. Non-negative conjugate gradients. arXiv:2607.22121, 2026.
- [10] J. J. Sylvester. A question in the geometry of situation. *Quarterly Journal of Pure and Applied Mathematics* 1:79, 1857.
- [11] M. Frank and P. Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly* 3:95–110, 1956.
- [12] E. Welzl. Smallest enclosing disks (balls and ellipsoids). *New Results and New Trends in Computer Science*, LNCS 555, 359–370, 1991.
- [13] P. Wolfe. Finding the nearest point in a polytope. *Mathematical Programming* 11:128–149, 1976.